



BUILD-YOUR-OWN-THING QUEST

YOU'RE THE MAESTRO

You have the ideas. A team of **robot coders** does the typing – one small job at a time, and you check every single one.

 about 30 minutes to start



no coding experience needed



we'll build a Tetris game (or your own idea)

Here's the whole plan

Three moves. That's it. Everything below is just those three moves, slowly.



1. You describe it

You answer a few questions about the thing you want to make. Short answers are fine.



2. It makes a plan

Your idea gets chopped into small jobs, in the order they should be built. You approve it.



3. It builds, you check

One small job per turn. You look at the result before the next one starts. You're in charge.



Word check – "the board". Your project's to-do list. Big goals are called **epics** (like "make the pieces fall"), and each small job is a **ticket** (like "make the left arrow key move the piece"). Robots work tickets, one at a time.



Get your gear

Tick each box as you go – your progress bar fills up.

You need three free tools installed. If they're already there, great – tick and move on. Installing things might need a grown-up's password.



Node.js, version 18 or newer

Type `node --version` in your terminal. If you see something like `v20.11.0`, you're set. No number? Install it from nodejs.org.



Git

Type `git --version`. Git is a time machine for your files – it remembers every version, so nothing is ever really lost.



Claude Code

The AI tool you'll type your wishes into. This is where the robot team lives.



Word check – "terminal". A window where you type commands instead of clicking buttons. On a Mac it's called **Terminal**; on Windows, **PowerShell**. Type a line, press Enter, and the computer does it. Typos are normal – just retype.



Make a home for your project

Three lines. Then the questions start.

Open your terminal and type these lines, pressing Enter after each one. Swap `tetris-clone` for whatever you're making.

 terminal

```
# make a new folder and step inside it
mkdir tetris-clone
cd tetris-clone

# bring in AI Maestro and start the questions
npx @mychiefmind/ai-maestro setup
```

LINE

WHAT IT ACTUALLY DOES

`mkdir tetris-clone`

Make directory – creates an empty folder for your project.

`cd tetris-clone`

Change directory – walks into that folder. Now you're standing inside it.

`npx ... setup`

Downloads AI Maestro, sets it up, and starts asking you about your idea.



It tidies up for you. Setup also starts Git for you (your time machine), creates the robot team, and checks everything is wired up correctly. You don't run any of that yourself.



Done – the questions appeared on my screen

You should see *"AI Maestro – tell me about the project"*. That's Level 2.

2

Describe your idea

Eight questions. Every one has an escape hatch.

This is the important bit – it's you telling the robots what you actually want. Answer in normal words. Nothing here is permanent.



The magic key is Enter. Every question shows a suggested answer in brackets. Press Enter to accept it. Most say `propose one`, which means *"you figure it out and show me"* – the robots will suggest an answer later and ask you to approve it. You can't get this wrong.

the questions

Project name (tetris-clone):

Product outcome – what the finished product should do (propose one):

Primary users – who it is for (propose one):

Stack – languages, frameworks, database, hosting (propose one):

Constraints – non-negotiable technical or product rules (propose one):

How it should run locally (propose one):

How it should be tested (propose one):

Areas of work (comma-separated) (backend, frontend, infra, docs):

Open the visual board now? (Y/n):

IT ASKS...	IT REALLY MEANS...
Product outcome	What can someone <i>do</i> when it's finished? "Play Tetris in a browser, with a score and levels."
Primary users	Who's going to use it? "Me and my friends, on phones and laptops."
Stack	What it's built out of. Don't know? Press Enter and let the robots pick.
Constraints	Your rules that can't be broken. "It must work on a phone." "Don't collect anyone's personal info."
How it should run	The command that starts your thing so you can look at it.
How it should be tested	How we prove it works – a robot must check its own homework.
Areas of work	The parts of the job. A browser game only needs <code>frontend, docs</code> .
Visual board	Say Y to open a colorful web page showing all your jobs. Press <code>Ctrl+C</code> to close it later.



Answer whatever you know, press Enter for the rest. Anything you skip gets written down as an **open question** – I'll come back with a suggestion and wait for your yes.



Done – it said my project is ready

You'll see a green  and a short list of next steps. Installing is finished. Really.



Ask for a plan

One command. Nothing gets built yet.

Open the AI tool in your project folder:

```
terminal
```

```
claude
```

Then type this and press Enter:

```
claude code
```

```
> /project-plan
```

Now the robots read your answers and turn them into a to-do list:



3–6 epics

The big goals. For Tetris: get it on screen → make it playable → make it fun.



5–15 tickets

The small jobs inside those goals, each one small enough to check in a few minutes.



An order

Some jobs must wait for others. You can't clear a line before pieces can fall.



It stops here on purpose. No code gets written yet. Planning ends with the plan on your screen and a question: *is this right?* That pause is yours.



Done – I can see my epics and tickets

Also shown: which job is ready first, and any guesses it wants you to confirm.

4

Be the boss

Read the plan. Change what you don't like.

This is the most Maestro-ish moment of the whole thing. Read it like you're checking someone else's homework.



Is everything I imagined in there?

If something's missing, say so.



Is each job small?

"Build the whole game" is too big. "Make pieces rotate" is right.



Can I tell when a job is done?

Each ticket should say what "finished" looks like.



Is something ready to start right now?

If everything is waiting on something else, nothing can begin.

Want changes? Just say them in normal words:

 claude code

```
> Change the plan: add a ticket for sound effects, split the
movement job into "move left and right" and "rotate", and
don't build anything yet.
```

Happy with it? Make a save point:

 terminal

```
git add .
git commit -m "chore: initialize project with AI Maestro plan"
```



Word check – "commit". A save point, exactly like in a game. Git remembers this moment forever, so you can always come back to it if a later change goes wrong.



Done – plan approved and saved

Now the fun part.

5

Build it, one job at a time

The command you'll use over and over.

● ● ● claude code

```
> /orchestrator
```

Here's what happens behind the curtain, every single time:

WHAT THE ROBOT TEAM DOES

- 1 Picks the most important job that isn't waiting on another one.
- 2 Copies your project into a safe sandbox, so your real files can't break.
- 3 Writes the code – a planner robot first, then a builder robot.
- 4 A different robot checks the work, like a teacher marking it.
- 5 If it passes, the work joins your real project. If not, it stops and tells you why.



Word check – "worktree". That safe sandbox: a separate copy of your project where a robot can make a mess without touching the version you care about. Nothing joins your real project until it passes the check.



I do **one job per turn**, then stop and wait. That's not me being slow – it's your turn to look. Type `/orchestrator` again for the next one.



Now repeat. Look at what changed, then run it again. Job by job, your idea turns into a real thing you can open and play.



Done – my first ticket is finished



You just conducted an AI team. Keep going.

Copy my homework: a Tetris clone

Stuck on what to type at Level 2? Use these answers exactly, and you'll be building Tetris.

● ● ● answers for a Tetris clone

Project name: tetris-clone

Product outcome: A browser Tetris game with a score, levels, pause, restart, and a game-over screen.

Primary users: Players on laptops and phones.

Stack: Plain JavaScript, HTML and CSS, drawing on a canvas.
No extra tools.

Constraints: Keep the game rules separate from the drawing code; work with both keyboard and touch; store nothing about players.

How it should run locally: npx serve

How it should be tested: Node's built-in test runner for the game rules, plus a quick play-test in the browser.

Areas of work: frontend, docs

A plan for that usually comes out looking like this:

EPIC (BIG GOAL)

TICKETS (SMALL JOBS), IN ORDER

Get it on screen

the empty game box → drawing the grid

Make it playable

the 7 piece shapes → moving and rotating → falling, landing, clearing lines

Make it fun

score and levels → pause, restart, game over, phone controls

When something goes wrong

It will. That's normal, and none of it breaks your computer.

YOU SEE...	WHAT TO DO
<code>command not found: npx</code>	Node.js isn't installed yet. Go back to Level 0.
<code>command not found: git</code>	Install Git, then try the same line again.
It says idle	Every job is waiting on another job. Say: <i>"nothing can start – fix the order in the plan"</i> .
It says blocked	A robot hit a real problem and stopped instead of guessing. Read the reason – that's the honest answer, not a failure.
The board page won't open	In your terminal: <code>cd maestro</code> then <code>npm run board</code> , and visit <code>localhost:5273</code> .
You changed your mind about the idea	Just say so in the AI tool: <i>"add a rule: it must work on a phone"</i> . Your brief gets updated.



You can always ask. The robots read plain English. "I don't understand what blocked means" is a perfectly good thing to type.

Cheat sheet

Everything you need, on one card.

TYPE THIS	WHERE	WHAT IT DOES
<code>npx @mychiefmind/ai-maestro setup</code>	terminal	Sets everything up and asks about your idea
<code>claude</code>	terminal	Opens the AI tool in your project
<code>/project-plan</code>	Claude Code	Turns your idea into epics and tickets, then stops for you
<code>/orchestrator</code>	Claude Code	Builds the next job – one per turn
<code>npm run board</code>	terminal, in <code>maestro</code>	Opens the colorful board of all your jobs
<code>git commit -m "..."</code>	terminal	Makes a save point you can always return to

NOW GO MAKE SOMETHING

You don't have to know how to code to lead a team that does.
But keep reading the work – a Maestro who never listens isn't conducting anything.